

Demo v6.2 流水线：23 个设计问题的代码答案

本文按设计评审中的 23 个问题，说明 Demo v6.2 每个阶段由哪个模块负责、数据如何流动，以及出错时在哪里终止。每个答案末尾都给出“源码证据”，引用格式为 文件::函数/类/方法；设计文档只作为语义补充，不替代实现证据。行号链接对应当前 single-camera 工作区源码。

整体运行结构如下：main.py::main 是总控入口，它启动 main_data_processing.py 子进程。后者由多个 mdp_demo_* mixin 组成，负责 摄像头、分割、跟踪和 warm-up。总控进程持续读取摄像头进程写出的 frames.jsonl，通过 chunk_* 模块生成在线 chunk，同时通过 shape_prior_* 模块生成 SAM3D shape prior，最后启动一个下游消费者：visualize_track.py，或 Phystwin_shen 的 scripts/run_online_full_pipeline.py supervisor。Demo 只直接管理这个 supervisor；Stage 1、可选 Stage 2、train 和一个合并 HTML viewer 由外部 wrapper 创建并继承同一个进程组。生命周期事件写入 pipeline_status.jsonl；Q23 会区分“已经实现的状态写入”和“默认 viewer 尚未显示的部分”。

摄像头与逐帧 I/O (Q1-Q6)

1. **摄像头在哪里启动？** 正式编排路径中，main.py::main 先调用 main_subprocess.build_main_data_processing_command，再用 subprocess.Popen 启动 main_data_processing.py。子进程入口构造 MainDataProcessingDemo；它继承的 _LifecycleMixin.run 调用 main_warmup.prepare_runtime_services_and_source，后者按输入模式选择 mdp_capture_source._start_realsense_pipeline (live) 或 RecordedRgbDFrameSource (fake-live)。

正式编排的设备由 config/default.yaml 中 camera.camera_serials 指定，默认是 ["239222300740"]。列表 schema 为以后多相机扩展保留，但 main_options.resolve_camera_serials 当前要求 恰好一个 非空 serial；可重复的 --camera-serial 会整体覆盖配置列表。main_subprocess.build_main_data_processing_command 将唯一 serial 传给 子进程 --serial，_start_realsense_pipeline 再执行 config.enable_device(serial)。table_calibrate_metadata.json 也记录该 serial 为 table-calibration reference。

这个“一台指定相机”的约束只覆盖由 main.py 启动的正式路径。若直接运行 main_data_processing.py 且不传 --serial，utils.camera.resolve_serial 仍会选择排序后的第一个 D400。此前 D405 缺少所需 RGB profile 的报错是实机验证结果，不是 Python 源码主动抛出的 固定错误。

源码证据：

- main.main → build_main_data_processing_command → main_data_processing.main → _LifecycleMixin.run。
- prepare_runtime_services_and_source 包含 replay/live 分支；_start_realsense_pipeline 绑定设备并启动 RealSense pipeline。
- resolve_camera_serials、main_cli.build_parser、camera.camera_serials 和 table_calibrate_metadata.json 共同 证明正式路径的 serial 合同。
- list_d400_serials 返回排序列表，resolve_serial 在未指定时取 serials[0]；实机 结果记录在 2026-07-09-demo-v6-2-refactor.md。

2. **摄像头线程在哪里创建？** `_LifecycleMixin._start_threads` 组装 worker 列表，并统一创建 `daemon=True` 的 `threading.Thread`。正式 strict 路径包含 `capture`、`seg`、`lossless PCD`、`lossless tracker` 和 `pair-output worker`。

`seg` 和 `lossless tracker` 会在同一个进程、同一个 CUDA device 上并发执行。EdgeTAM 的 `reduce-overhead` 会在第 2 次 `model call` 录制 CUDA graph；TAPNext++ 第一次拿到 `mask` 时才构造 CUDA model，并在参数初始化时使用 RNG。为避免两者重叠，`live` 与 `replay` 都会在发布 frame 0 后等待完整的首个 PCD/tracker pair，再释放 frame 1。这个启动 `handshake` 保留后续并发与 `compile` 性能，同时满足 CUDA graph `capture` 期间不能有其他线程 CUDA 工作的约束。`_capture_recording_worker` **不是另一条线程**；`replay` 模式下，`capture` 线程进入 `_capture_worker` 后同步调用它。

`shape-prior warm-up` 另由 `ShapePriorWarmupManager.maybe_submit` 创建一条 `shape-prior-warmup daemon thread`；它不在 `_LifecycleMixin._threads` 列表中。

源码证据：

- `_LifecycleMixin._start_threads` 定义 worker targets，并在一个循环里调用 `threading.Thread(..., daemon=True)`。
- `_CaptureMixin._capture_worker` 的 `replay` 分支 直接调用 `_capture_recording_worker` 后返回。
- `ShapePriorWarmupManager.maybe_submit` 单独创建 `shape-prior warm-up` 线程。

3. **进程和线程如何协作？** 外层是多进程：总控进程启动 `camera` 子进程；可选 `visualizer`、`Phystwin_shen full-pipeline supervisor` 也是总控的直接子进程。`supervisor` 再按配置启动一个合并 `HTML viewer`、`Stage 1`、可选 `Stage 2` 和 `train`；这些 `child` 不创建新 session，因此继承 `supervisor` 的进程组。`shape-prior` 不是一条与 `camera` 并列、常驻的单一进程：`camera` 进程先启动 `warm-up thread`，`ShapePriorLocalClient.request_shape_prior` 再顺序调用各阶段子进程。

`camera` 进程内部的 strict 数据流是：`capture` 同时写 `capture_slot` 和 `lossless_frame_queue`；`seg` 将 `mask` 写到 `PCD/tracker` 两条有序队列；`PCD` 与 `tracker` 结果由 `SameSeqPairer` 按相同 `seq` 配对，再进入 `pair-output queue`。`OrderedPacketQueue` 保序且不静默覆盖，`LatestSlot` 只保留最新值。`stop_event` 管理 `_LifecycleMixin._threads` 中的 `workers`；`shape-prior manager thread` 不检查该 event，因此不能笼统说“所有线程都由 `stop_event` 退出”。

源码证据：

- `main.main`、`launch_phystwin_shen` 和 `ShapePriorLocalClient.request_shape_prior` 给出 Demo 侧进程边界；`full-pipeline wrapper` 的 `child` 顺序由外部 `scripts/run_online_full_pipeline.py` 定义；`shape-prior` 阶段冷启动由 `_run_stage` 调用 `subprocess.run`。
- `_publish_capture_packet`、`OrderedPacketQueue`、`SameSeqPairer` 和 `LatestSlot` 给出实际的线程间数据合同。
- `_LifecycleMixin.stop` 设置 `stop_event` 并 `join _threads`；`shape-prior thread` 的入口是 `ShapePriorWarmupManager._run`。

4. **RealSense RGB 和 depth 的 FPS 是多少？** 由 `main.py` 启动的正式路径默认以 **30 FPS** 采集 `RealSense`。`build_main_data_processing_command` 把外层 `--camera-fps` 传成子进程 `--fps`，`_start_realsense_pipeline` 对所有启用的 `color`、`IR` 或 `depth stream` 都使用同一个 `int(args.fps)`。

strict `live` 输出采样由 `LiveLatestFrameSampler` 控制：直接输入是子进程的 `--lossless-input-fps`，外层由 `resolve_camera_source_replay_fps` 解析。未单独设置 `--camera-source-replay-fps` 时它回落到 `--replay-fps`，默认 **5 FPS**；设置 `override` 后，`camera` 取样频率可以与 `chunk/metadata` 使用的

replay_fps 分开。采样器每隔 1/fps 发布当时的最新帧。直接运行 main_data_processing.py 的内层 -fps 默认仍是 60，因此 30 FPS 结论应限定为正式 main.py 编排默认。

这层采集频率不等于发布/物理频率。Demo 的 replay_fps 固定输出 **5 FPS** 均匀时间步；Phystwin_shen configs/real.yaml 同样使用 FPS: 5、dt: 5e-5、num_substeps: 4000，即每个数据帧模拟 $5e-5 \times 4000 = 0.2 \text{ s} = 1/5 \text{ s}$ 。因此不存在原先 30 FPS 物理时间比 Demo 快 6 倍的问题；RealSense 仍可在 30 FPS 获取最新输入。

源码证据：

- config/default.yaml 定义 replay_fps: 5.0，camera.camera_fps 定义 camera_fps: 30；build_main_data_processing_command 将后者传给子进程。
- _start_realsense_pipeline 对每个启用 stream 都传 int(args.fps)。
- resolve_camera_source_replay_fps、_CaptureMixin._capture_worker 和 LiveLatestFrameSampler 证明“固定 tick 取最新帧”的实际控制链。
- mdp_cli.build_parser 与 mdp_constants.DEFAULT_FPS 证明直接运行子入口的默认值是 60，而不是 30。

5. **每帧在哪里读取？** live 模式由 _CaptureMixin._capture_worker 调用 pipeline.wait_for_frames()；native-depth 路径先 align.process，FFS 路径读取 color 与左右 IR。replay 模式由 _capture_recording_worker 调用 RecordedRgbFrameSource.read_packet 加载录制 RGB-D/IR 文件。两条路径都生成 mdp_packets.FramePacket。

_publish_capture_packet 总会写 capture_slot；正式 strict 模式还会同时写 lossless_frame_queue，不是二选一。

源码证据：

- _CaptureMixin._capture_worker 包含 RealSense 读取、对齐、数组复制和 FramePacket 构造。
- _capture_recording_worker → RecordedRgbFrameSource.read_packet 是 replay 读取链。
- FramePacket 定义逐帧包；_publish_capture_packet 明确先写 slot，再在 lossless 模式写 queue。

6. **frame id 和 timestamp 从哪里来？** FramePacket.seq 是 Demo 内部逻辑序号。live 路径用 output_seq 从 0 连续重编号；replay 路径把 runtime_seq 作为内部 seq。

fake-live 的来源字段更细：source_timestamp_s 来自录制 metadata 的 value；source_frame_index 是排序后 frame refs 中被选中的位置；原始 metadata key 则保存在 source_step。当前 live 实现没有调用 RealSense get_timestamp()/get_frame_number()，所以 live FramePacket 的三个 source_* 字段保持 None。原文“真实采集模式下来自摄像头”不受当前源码支持。

有值的 source_* 会一路写入 prepared frame 和 archive mapping。正式均匀时间轴使用连续 online/output index 除以 fps，而不是 raw source_frame_index / fps；source_* 只做 provenance。

源码证据：

- _CaptureMixin._capture_worker 的 publish_output_packet 连续重编号，且 live FramePacket(...) 未赋 source_*。
- RecordedRgbFrameSource._build_frame_refs 与 read_packet 分别构造 step/timestamp，再写 source_timestamp_s/source_frame_index/source_step。

- `HeadlessCaptureWriter.write_pcd` → `prepare_phystwin_frame` 保留 provenance; `OnlineFrameArchive.archive_chunk` 写 online-to-source mapping。
- `ChunkDataWriter.commit_chunk_data_record` 维护连续 `start_frame/end_frame`; `_write_chunk_from_rows` 用 `row_start/fps` 和 `row_end/fps` 生成均匀窗口时间。

Warm-up (Q7–Q14)

7. **Warm-up 使用单帧还是一段视频?** 初始化使用一张帧。`prepare_segmentation_warmup` 取得一张 `first_frame`, `resolve_initial_mask_bundle` 只把这张图交给 SAM3.1, 生成 object、hand A、hand B 的 `InitialMaskBundle`。随后 `_seg_worker` 创建一个 EdgeTAM session, 并对同一 `first_frame` 调用 `_run_segmentation_frame(..., add_prompt=True)`。这是一个 session 中的三个 identity, 不是三个 session。

源码证据:

- `prepare_segmentation_warmup` → `resolve_initial_mask_bundle` → `run_sam31_first_frame_mask_bundle` 只处理 `first_frame.color_bgr`。
- `_SegWarmupMixin._seg_worker` 创建一个 `EdgeTamVideoInferenceSession`, 再调用 `_run_segmentation_frame` 并设置 `add_prompt=True`。

8. **系统如何确认拿到的是 frame 0?** 正式 strict 路径不是靠 sentinel 单独保证, 而是靠 producer handshake 加 FIFO: live 首次发布时 `output_seq == 0`, replay 首次显式调用 `read_packet(seq=0)`; 两者在发布首帧后都会等待 `_first_frame_segmented`, 随后继续等待 `_lossless_first_pair_published`, 不会先把后续正式帧灌入。`strict_wait_for_first_frame` 从 `OrderedPacketQueue` 队首取帧, 因此得到 `seq 0`。

非 strict 分支才调用 `capture_slot.get_latest_after(-1)`; 它只保证返回 `seq > -1` 的当前最新帧, 不能单独证明一定是 `seq 0`。首帧只“添加 prompt 一次”, 但它还会继续用于 PCD、tracker、shape prior 和 warm-up anchor, 不能说整张帧只用于初始化。

源码证据:

- `_wait_for_first_frame` 展示 strict queue 与 non-strict latest-slot 两条分支。
- `_CaptureMixin._capture_worker` 和 `_capture_recording_worker` 都先发布 `seq 0`, 再等待首帧分割 handshake。
- `OrderedPacketQueue.put/get` 保持连续 FIFO; `LatestSlot.get_latest_after` 则是 latest-wins。
- `_SegWarmupMixin._seg_worker` 对首帧用 `add_prompt=True`, 后续帧统一用 `False`。

9. **Warm-up 期间后续到达的帧如何处理?** 后续帧继续进入同一个 EdgeTAM session, 使用 `add_prompt=False`, 并继续生成 mask、PCD、tracker 结果和 input preview。只有正式 product row 受 gate 控制: `chunk-ready anchor` 已写且 `shape-prior` 仍为 pending/running 时, `_formal_chunk_rows_gated` 扣留 `frames.jsonl row` 和对应 tracker sidecar。

“期间 `frames.jsonl` 只有 frame 0” 需要限定: anchor 前可能先写入无效 startup rows, chunk bridge 随后会修剪; gate 超时或 shape-prior 进入终态后也会解除。因此这句话只适用于正常的“anchor 已写、prior 仍在运行、gate 未超时”区间。

源码证据:

- `_SegWarmupMixin._seg_worker` 对后续帧调用 `_run_segmentation_frame(..., add_prompt=False)` 后仍发布 mask。
- `_PairPublishMixin._publish_strict_render_pair` 在 PCD/tracker 已配对后只对落盘行应用 gate; `_formal_chunk_rows_gated` 定义其状态条件。
- `_PcdMixin._write_headless_pcd_result` 在 gated 时 返回; `HeadlessCaptureWriter.write_input_frame` 不经过该 gate。
- `_trim_warmup_delayed_rows` 修剪无效 startup rows; `_headless_product_rows_gated` 实现超时解除逻辑。

10. **Warm-up 最耗时的步骤是什么？** 权威产物是 `capture/shape_prior_profile.json`。现在它把操作员等待时间拆成 三层，而不是只报告 submit 后的五个粗阶段：

1. `shape_prior_timing.pre_submit`: camera runtime start → frame 0 receive → EdgeTAM mask ready → PCD ready → shape-prior submit。它同时保留 frame-0 SAM3.1、EdgeTAM 和 PCD 的已有细分 timing。
2. `shape_prior_timing.critical_path`: case write → upscale → SAM3.1 RGBA export → SAM3D generate → align → sample → result finalize。每项有 start/end offset、wall duration 和子阶段 details。
3. 顶层 `warmup_shape_prior_ready_to_gate_open_ms` 与 `warmup_total_ms`：记录 prior ready 后写 capture 产物并真正打开 formal gate 的延迟，以及从 camera runtime start 到 gate open 的总等待。

当 shape-prior 产物已经写入且 formal gate 首次打开时，camera 子进程会在 当前终端只打印一次醒目的 Warmup finished banner。它和 `STAGE_WARMUP_READY` 在同一个真实完成边界触发，不会在 submit 或后台计算 尚未完成时提前显示。

`shape_prior_timing.ranking` 按关键路径 wall duration 排序，bottleneck 是本轮第一优化目标；`accounted_ms` 和 `unattributed_ms` 用于检查计时是否 闭合。这里的排名只描述本轮，不能用单次结果代替多轮 p50/p95。

三个预热子进程还会在 `<shape-prior-case>/shape/timing/{upscale,generate,align}.json` 写 READY 与 completed 快照；sample 写 `sample.json`。父进程比较 READY wall time 与 GO wall time，给出 `ready_before_go`、`ready_lead_ms` 和 `startup_tail_on_critical_path_ms`，因此可以区分“模型已经预热完成”和“GO 发出后仍在补模型加载”。`profile_snapshot_to_parent_return_ms` 还把最后一次 profile snapshot 后的 JSON flush/进程退出成本单独暴露出来。

子阶段拆分直接对应可优化动作：upscale 区分 model load/crop/inference/ PNG write；generate 区分 prepare/model load/pipeline run/GLB/PLY export；align 区分 input/render candidates/SuperGlue/PnP+scale/两段 ARAP/mesh export；sample 区分 mesh load/surface+volume sample/voxel dedup/pickle。

generate 的 SAM3D 调用固定关闭 `with_layout_postprocess`：Demo v6.2 只消费 GLB mesh 和 Gaussian，不使用 SAM3D 返回的 layout/pose 字段；随后独立的 `shape_prior_align` 才负责把 mesh 配准到 frame-0 观测。因而 layout pose 后处理不会决定最终对齐，只会增加 warm-up 关键路径。mesh postprocess 与 texture baking 仍保持启用，分别服务后续 mesh 对齐/采样和带纹理 GLB 导出。

修改前保存的一轮基线中，submit 后总计约 59.23 s：generate 29.44 s（49.7%）、align 14.26 s（24.1%）、upscale 12.27 s（20.7%）；三者合计 94.5%。camera start 到 submit 另约 15.94 s。因此下一轮详细 profile 应先判断 generate 的模型加载还是 pipeline/export 最慢，再判断 align 的

render/match 与 ARAP，占比很小的二次 SAM3.1 segment 不是首要目标。

源码与实测证据：

- `shape_prior_timing` 定义 schema 校验、关键路径 闭合与 bottleneck 排名；`ShapePriorLocalClient.request_shape_prior` 聚合它。
- `run_sam3d_shape_prior`、`shape_prior_align.main`、`shape_prior_sample.main` 和 `image_upscale.main` 写子进程明细。
- `outputs_v6_1/capture/shape_prior_profile.json` 是修改前 59.23 s 基线；需要下一次运行才会生成新的详细 schema。

11. Warm-up 完成后保留哪些状态和文件？ `_seg_worker` 在其生命周期内保留

`SegmentationWarmupState`、`InitialMaskBundle` 和唯一的 EdgeTAM session；三个 identity 在同一次 `add_inputs_to_inference_session` 中注册。`shape-prior manager` 成功后还在 `self._result` 中保留 `ShapePriorResult`。

磁盘上会保留 offline-style shape-prior case、配置路径下的 `shape_prior/points.npz`、capture 下的 `shape_prior/points.npz`、`<case>/shape/matching/final_mesh.glb`，以及 shape-prior sampling 生成的 `<case>/final_data.pkl`。

源码证据：

- `InitialMaskBundle` 与 `SegmentationWarmupState` 的实例由 `_SegWarmupMixin._seg_worker` 持有；同一 worker 只构造一个 session。
- `ShapePriorWarmupManager._run` 成功时写 `self._result`，`ready_result` 读取它。
- `write_shape_prior_case` 与 `ShapePriorLocalClient.request_shape_prior` 写 case 和配置的 `points.npz`；`HeadlessCaptureWriter.write_shape_prior_result` 写 capture 副本。
- `shape_prior_align.main` 导出 `final_mesh.glb`；`shape_prior_sample.main` 写 case `final_data.pkl`。

12. Warm-up 状态如何校验？ `resolve_initial_mask_bundle` 校验 controller/object mask 与输入帧尺寸；`_union_masks` 拒绝“没有任何 instance mask”和同一 label 内形状不一致，但不会把“一张全 false mask”自动视为缺失。`split_controller_hand_instances` 必须得到两只非空、可分离的手。

`write_shape_prior_case` 校验 RGB/depth/mask 形状，拒绝空 object mask，并在 radius 清理后没有有效 object depth 点时失败。当前源码没有通用的“点数不足”阈值；controller 没有有效点时甚至会借用一个 object point。`chunk` 边界只要求 surface+interior 总数大于 0。因此原文应收窄为“完全没有有效 object depth 点或 shape-prior 点时失败”。

源码证据：

- `resolve_initial_mask_bundle`、`_union_masks` 和 `split_controller_hand_instances` 给出 frame-0 mask 校验。
- `write_shape_prior_case` 调用 `_as_mask`，检查 object_mask 与有效 depth，并在 controller 空时执行 `controller_points = object_points[:1].copy()`。
- `_shape_points_for_chunk` 只检查 surface+interior 总数是否大于 0；不存在更高的通用最小点数门槛。

13. Warm-up 出错后如何处理？冷启动 shape-prior stage 由 `_run_stage` 使用 `subprocess.run(..., check=True)`；预热 worker 也会在非零 return code 时抛 `CalledProcessError`。但 shape-prior 总控本身是 camera 进程内的 daemon thread；`ShapePriorWarmupManager._run` 会捕获 stage 异常，将

profile 置为 failed，而不是直接走 camera worker 的 fatal hook。

segmentation 等正式 workers 的异常会进入 `_record_fatal_worker_error`：记录第一条 fatal、写 fatal_error 状态、设置 stop_event，最后让 `MainDataProcessingDemo.run` 返回 2。shape-prior 失败时，status 不再是 pending/running，row gate 解除；失败 profile 写入 capture metadata，chunk bridge 的 `_shape_points_for_chunk` 看到 failed 后立即抛错，避免无限等待。

源码证据：

- `_run_stage` 和 `_run_prewarmed_stage` 给出子阶段非零退出的 异常路径。
- `ShapePriorWarmupManager.maybe_submit` 创建 thread；`ShapePriorWarmupManager._run` 捕获 异常并写 STATUS_FAILED。
- `_record_fatal_worker_error` 与 `_LifecycleMixin.run` 给出 camera worker 的 fatal/exit 路径。
- `_formal_chunk_rows_gated`、`_maybe_write_shape_prior_headless_result` 和 `_shape_points_for_chunk` 给出 shape-prior failed → metadata → bridge exception 的真实传播链。

14. **正式时间线从哪里开始？** 成功路径中，第一个 chunk-ready row 占据 online/output frame 0 的 warm-up anchor；anchor 后、shape prior 尚为 pending/running 的 rows 被扣留。shape prior 进入 READY 后第一条未被 gate 的 row 紧接为 online frame 1。OnlineFrameArchive 连续编号，同时在 enhance_metadata.json 保留原 seq 和 source mapping。

这里也有失败边界：gate 会在 timeout 或 failed/disabled 终态解除，所以 “READY 后成为 frame 1” 只描述成功路径；失败路径随后应由 bridge 报错。

源码证据：

- `_formal_chunk_rows_gated` 定义 anchor 后的 gate；`_PcdMixin._write_headless_pcd_result` 只让 controller ≥ 30 且 object > 0 的 row 取得 anchor，并记录首次 formal seq。
- `_trim_warmup_delayed_rows` 保留首个 chunk-ready row；`OnlineFrameArchive.archive_chunk` 用 online_start_frame + local_index 连续编号。
- `design_spec.md` 记录 frame 0/1 接缝与 hold-still 约定，但实现依据是上述函数。

Chunk 组装、跟踪与过滤（Q15–Q19）

15. **Chunk 如何组装？** `stream_chunk_data_from_headless_capture` 每读到一条 frames.jsonl row，就用 `_prepared_frame_from_row` 加载它引用的 canonical prepared NPZ，并将 row/frame 同步追加到两个 buffer。达到 chunk_size 后先形成 pending_window；下一条 row 通常作为 borrow/lookahead frame 到达后才 materialize，capture 结束时则无 borrow flush。borrow 只参与上一窗口尾帧的 motion verdict，不进入其发布数组。

`_chunk_data_window_from_prepared_frames` 校验所有帧共享 query schema，逐帧收集 processed mask，并将 track、visibility、PCD points/colors 沿时间维 stack。RGB 不在此处 stack，而是稍后由 `OnlineFrameArchive.archive_chunk` 逐帧写 PNG；`query_points_yx` 只做共享一致性校验。当前路径不会从 legacy sidecar 重建数据。

源码证据：

- `stream_chunk_data_from_headless_capture` 负责 row/frame 双 buffer、pending_window 和 borrow-row 触发；其嵌套 `_materialize_pending` 定义在同函数内。

- `_prepared_frame_from_row` 要求 `prepared_phystwin_frame_path`; 缺失或文件不存在立即失败。
- `_chunk_data_window_from_prepared_frames` 检查 query 一致性、收集 mask, 并 stack track/visibility/PCD; `_write_chunk_from_rows` 将 RGB-D 交给 archive。

16. **Chunk 大小在哪里配置?** `main_options.resolve_chunk_frame_count` 优先使用显式 `--chunk-frame-count`; 否则计算 `round(replay_fps × chunk_seconds)`, 并要求结果大于 0。默认配置为 5 FPS × 7 秒 = 35 帧。结果传给 chunk stream, 并保存为 `ChunkDataWriter.chunk_size`; writer 再次校验正数, 并把它写入 manifest 和 static metadata。

源码证据:

- `config/default.yaml` 定义 5 FPS, `chunking.chunk_seconds` 定义 7 秒; `main_cli.build_parser` 暴露 override。
- `resolve_chunk_frame_count` 实现 override/乘法/正数 校验; `main.main` 把结果传入 chunk stream。
- `ChunkDataWriter.__init__` 再次校验并保存 `self.chunk_size`; `_write_manifest` 发布它。

17. **Chunk 按时间还是按帧数关闭?** 窗口边界严格按 **row/frame 数量**: 每次 append 后, buffer 未满就 continue, 达到 `chunk_size` 才关闭。`window_closed_wall_s` 只是遥测, 不参与边界判断。live 路径关闭完整窗口后, 通常仍需等下一帧作为 borrow 才 materialize/commit, 因此“按帧数关闭”不等于“第 35 帧到达即发布”。

源码证据:

- `stream_chunk_data_from_headless_capture` 用 `len(row_buffer) < chunk_size` 判断是否继续累积。
- 该函数中的 `pending_window` 与 `_materialize_pending` 证明 borrow-row 发布延迟; `_write_chunk_from_rows` 只把 `window_closed_wall_s` 写入 telemetry。

18. **Chunk 组装后如何执行跟踪?** 唯一的实时 chunk-stream 入口为整个 session 创建一次 `tracking.TrackingRuntime`, 并把同一实例传过 `_write_chunk_from_rows` 到 `_chunk_data_window_from_prepared_frames`。后者调用 `_track_input_with_session_query_schema` → `tracking.build_window_observations` → `TrackingRuntime.process_window`。

chunk 0 的 `_freeze_identity` 冻结 controller anchors、object columns、query schema 和 neighbor table; 后续窗口由 `_check_frozen_identity` 校验。process_window 用 `~ctrl_usable` 表示语义上的 temporary-invalid (没有单独命名的状态数组), 再调用 `_recover_anchor` 和 `Kabsch_rigid_transform` 做局部刚性恢复。若单独调用 window builder 而不传 runtime, 它会临时新建实例; “整个 session 一个 runtime” 只保证在公开 stream 主路径中成立。

源码证据:

- `stream_chunk_data_from_headless_capture` 在循环外创建一个 `TrackingRuntime`。
- `_track_input_with_session_query_schema` 调用 `build_window_observations`; `_chunk_data_window_from_prepared_frames` 调用 `TrackingRuntime.process_window`。
- `_freeze_identity`、`_check_frozen_identity`、`_recover_anchor` 和 `_rigid_transform` 给出冻结与恢复细节。

19. **跟踪后还会做些什么过滤?** `tracking.motion_consistency` 执行动作一致性过滤: 半径 0.01 m、至少 5 个邻居 (radius query 未排除自身)、相似阈值 0.005 m, 至少 50% 邻居同意。

depth-validity **mask refinement** 与 3D radius-outlier mask refinement 在 camera 写 canonical prepared frame 时按顺序执行一次；chunk 侧只加载结果，不再次做 radius refinement。不过 tracking.build_window_observations 仍会在 track pixel 采样时重新计算逐 query 的 depth-valid，因此不能泛称“所有 depth-validity 判断只执行一次”。

tracking 之后，AsapRuntime.augment_window 以 visibility、motion validity、finite 和 nonzero 联合判定无效 object 条目并回填。_deform_frame 在约束太少或结果非有限时复用上一帧 mesh vertices，这就是 design_spec_v6_1.md 所保留的 silent-freeze 行为。

源码证据：

- tracking.py 常量 与 motion_consistency 给出 0.01/5/0.005/50% 规则。
- prepare_phystwin_frame 依次调用 apply_depth_validity_to_mask_frame 和 apply_radius_outlier_to_mask_frame；build_window_observations 另做 query-level depth-valid。
- AsapRuntime.augment_window 计算 valid_now 并回填；AsapRuntime._deform_frame 实现 silent freeze。

训练侧 schema、manifest 与读取起点 (Q20–Q22)

20. 训练侧会收到什么数据？ Demo 生产端把每个窗口写成 online_data/chunks/chunk_{id:06d}.pkl。固定 metadata 包括 case_name、chunk_id、start_frame、end_frame、source_frame_indices；source_timestamps_s 仅在有价值时写入。data_keys.REQUIRED_TIME_KEYS 定义五个生产端必需时序键：object_points、object_colors、object_visibilities、object_motions_valid、controller_points。生产端通用 schema 把 asap_surface_points、asap_interior_points、controller_proxied 等列为 optional；并不存在名为 recovery_mask 的字段。

但当前配置的 Phystwin_shen consumer 会把 asap_surface_points/asap_interior_points 提升为必需字段；默认 asap_augment=True 会生产它们。因此“ASAP 可选”只描述 Demo writer 的通用 schema，不描述当前 trainer 的更窄合同。

同一 session 还生成 online_data/color/0/{k}.png、online_data/depth/0/{k}.npy (uint16 mm)、calibrate.pkl、metadata.json、enhance_metadata.json，以及前缀聚合后的 data/final_data.pkl。

源码证据：

- REQUIRED_TIME_KEYS/OPTIONAL_TIME_KEYS 定义生产端键；build_chunk_data_record 定义 chunk metadata 与 TIME_KEYS 切片。
- ChunkDataWriter.commit_chunk_data_record 写 chunk_{id:06d}.pkl；_append_static_data 聚合 data/final_data.pkl。
- OnlineFrameArchive._archive_one_frame、_initialize_calibration、_write_metadata 和 _write_enhance_metadata 给出 RGB-D archive 布局。
- 当前外部 checkout 5b8c071 的 OnlineFrameBuffer._validate_chunk_shapes 将两个 ASAP key 列为 required；main_cli.build_parser 把 asap_augment 默认设为 true。

21. Manifest 何时更新，如何保证读者不会看到半成品？ 正常提交顺序是：

OnlineFrameArchive.archive_chunk 先写本 chunk 的 RGB-D 帧文件；

ChunkDataWriter.commit_chunk_data_record 再原子写 chunk pickle，原子更新聚合

final_data/metadata, 最后原子更新 online_data/manifest.json; commit 返回后才调用 OnlineFrameArchive.publish_metadata 推进 archive 的 metadata.json 与 enhance_metadata.json。所以读者一旦从 manifest 看到 new committed chunk, 对应 chunk 与帧文件已经存在; archive frame_num 只推进到已 commit 前缀。

atomic_pickle_dump/atomic_json_dump 都采用临时文件、flush、fsync 和 os.replace。RGB PNG 也 fsync 后 replace; depth NPY 使用 temp+replace 但没有显式 fsync, 所以源码保证“不会看到半写文件”, 不能夸大成“所有 archive 文件都具备相同的断电持久性”。正常结束写 finished; materialize/commit try 块内的失败写 failed。更早的 prepared-frame 加载失败不在该 try 块内, 因此不能声称任何异常都必然把 manifest 从 recording 改成 failed。

源码证据:

- `_write_chunk_from_rows` 明确执行 `archive_chunk` → `commit_chunk_data` → `publish_metadata`。
- `ChunkDataWriter.commit_chunk_data_record` 的顺序是 `atomic chunk` → `aggregate` → `counters` → `_write_manifest`。
- `OnlineFrameArchive.archive_chunk` 与 `publish_metadata` 说明 frame files 与 committed metadata 的关系。
- `atomic_pickle_dump/atomic_json_dump` 和 `_atomic_write_bytes` 给出原子写细节; `stream_chunk_data_from_headless_capture` 给出 finished/failed 的实际覆盖边界。

22. 训练侧从什么时候开始读取? main.main 内嵌的 `_maybe_start_phystwin_shen` 在 warm-up 启用时等待 shape_prior/points.npz 出现后启动; warm-up 禁用时以 `warmup_disabled_immediate` 立即启动。Demo 只启动一个 `scripts/run_online_full_pipeline.py` supervisor, 并显式传入 `--online_dir <base_path>/online_data` 以及本地 `config/default.yaml::phystwin_shen` 的每个 runtime 叶子。外部 pipeline YAML 不再是 Demo 参数的维护源。Stage 1/2 各自的 `max_online_chunks`、`cma_popsizes`、`zero_order_backend` 和 `sim_force_mode` 也由本地 YAML 显式传入; 当前 Stage 1 是 2/4/boba/gather, Stage 2 是 10/4/boba/gather。batch_size/segment_len/segment_stride 遵循 Phystwin 原生的 common-then-stage 继承语义, 并通过 stage-specific CLI 显式传递。当前 common 不提供这三个默认值, Stage 1 使用 2/10/10, train 使用 5/30/30; 禁用的 Stage 2 可省略, 若启用且没有自身值或 common 默认值, Demo 会在 camera 启动前失败。

supervisor 在 demo_2_max 中运行; 外部 YAML 的 python: null 令 Stage 1、Stage 2、train 和 viewer 继承 supervisor 的同一个 Python。wrapper 先启动 `cma_viewer.source=all` 的单个合并 viewer, 再顺序运行 Stage 1、可选 Stage 2 和 train; 独立 train_viewer 保持关闭。Demo 在启动 supervisor 前只清理这个 viewer 的 endpoint, 默认是 127.0.0.1:8765。

supervisor、Stage 1/2 和 train 的合并 stdout/stderr 同时实时转发到 Demo 启动终端 (每行前缀 [phystwin_shen]) 并保留到 `<base_path>/phystwin_shen/online_full_pipeline.log`。Demo 显式设置 `PYTHONUNBUFFERED=1`, 避免外部 Python stage 因 pipe 重定向延迟输出。当前默认 viewer 使用 `--quiet`, 它们自己的输出仍由外部 wrapper 静默处理。

顺序读取逻辑位于外部 Phystwin_shen checkout: `OnlineChunkReader.load_new_chunks` 从 `last_loaded_chunk + 1` 顺序读到 manifest 的 `latest_committed_chunk`; `wait_for_initial_frames` 会先等到至少各 stage 自己的 `segment_len` 帧才创建 simulator/trainer。当前 chunk 是 5 帧, Stage 1 需要 10 帧 (2 chunks), train 需要 30 帧 (6 chunks)。train.stop_when_finished: true 时, trainer 以 manifest 的 finished 为停止条件, 完成并保存观察到 finished 的 terminal iteration; 设为 false 时严格跑 iterations。无论是哪一种, Demo 正常路径最终都会等待 supervisor 返回, 只有 return code 0 才算完整 pipeline 成功。

源码证据：

- `main.main::_maybe_start_phystwin_shen` 给出 `points-ready/disabled` 两个 trigger；
`launch_phystwin_shen` 只执行一次 `Popen`，`build_full_pipeline_command` 给出完整显式 CLI。
- 外部 checkout 的 `train_online_warp.py::wait_for_initial_frames`、
`qqt/data/online_stream.py::OnlineChunkReader.load_new_chunks` 和
`qqt/engine/trainer_warp.py::InvPhyTrainerWarp.train_online_batched` 给出等待、顺序读取、terminal save 和 stop-when-finished 行为。

在线流水线状态可视化（Q23）

23. 如何看到流水线当前在做什么，以及 warm-up 是否失败？`PipelineStatusWriter.emit` 把 `t/source/stage/detail/ok` 追加到 `<base_path>/pipeline_status.jsonl`，并吞掉所有写入异常，所以 telemetry 失败不会改变正式产品。当前实际 writer 只有两类：orchestrator 写 run start、chunk committed、Phystwin downstream start 和正常控制流末尾的 finished/fatal；camera 写 capture start、shape-prior submitted、warm-up ready，以及 startup/worker fatal。shape-prior 的 upscale/generate/align/sample 子进程没有各自创建 writer，也没有逐阶段状态事件。

`viz_playback.run_side_by_side` 会读取最近 200 条事件，并调用 `viz_panels.draw_pipeline_status` 绘制状态条；任一 `ok=False` 或 `STAGE_FATAL` 事件会令状态条变红并显示错误。但当前默认配置是 `side-by-side + sam3d-final-data`，`visualize_track.run` 会分派到 `run_interactive_side_by_side`；该函数目前没有读取状态日志，也没有绘制状态条。因此当前可核验的查看方式是：直接查看 `pipeline_status.jsonl`，或使用会进入 OpenCV `run_side_by_side` 的 `rgb-overlay` 路径；不能声称默认正式 viewer 已经显示该状态条。

`main.main` 的外层 `try/except/finally` 覆盖 camera 启动、stream、supervisor launch 和最终 wait。shape-prior/materialization、camera、supervisor 非零退出或 Ctrl+C 任一失败都会写 terminal `STAGE_FATAL`，并以保存的 PGID 对整个 Phystwin_shen 进程组执行 `SIGTERM`，超时后 `SIGKILL`。

显式 `--max-chunks N` 达标时，stream 已经提交完第 N 个 chunk 并持久化 terminal `manifest.status=finished`。编排器会在等待 Phystwin_shen 继续训练之前立即、只打印一次：

```
#####  
collect finish  
#####
```

这个 banner 只表示采集完成，不表示整个 Demo 进程已经退出；Phystwin_shen 尚未训练完时，主进程会继续等待它。

Warm-up 实时 RGB 输入预览 (`mdp_warmup_preview.WarmupRgbPreview`，不是 `tracking-chunk viewer`)：无论 `downstream.mode` 选什么，camera 进程在 capture 启动时打开一个实时 RGB 输入窗口（直接读内存里的 `input_preview_slot`，零磁盘 IO），供操作员在 hold-still 期间确认取景/双手可见。生命周期：warm-up 正常结束（`WARMUP_FINISHED` banner 处；若 shape-prior warm-up 关闭则在 `frame-0 seed` 完成处）自动关闭；warm-up 失败/取消/提前退出经 `stop_event + stop()` 立即关闭。GUI 失败（无显示环境）只打一行日志并禁用，绝不影响采集。开关：`--warmup-rgb-preview / --no-warmup-rgb-preview`（默认开，编排器透传给 camera 子进程）。即使 supervisor leader 已退

出，遗留的 train/viewer child 仍按保存 PGID 清理。reader 仍只把 manifest.status=finished 当完成；若 producer 写 failed，Demo 不等待 reader 自己理解 failed，而是由上述异常路径终止整个 downstream group。

源码证据：

- `PipelineStatusWriter.emit` 是 best-effort append；`read_status_events` 容忍缺失文件和 torn last line。
- `main.main`、`_LifecycleMixin.run/_record_fatal_worker_error` 和 `_maybe_start_shape_prior_from_pcd_result` 是全仓实际 emit call sites；shape-prior stage 文件没有 writer call site。
- `viz_playback.run_side_by_side` 调用 `draw_pipeline_status`；`use_interactive_side_by_side` 与 `run_interactive_side_by_side` 证明默认 SAM3D viewer 绕过该绘制逻辑。
- `config/default.yaml` 定义默认 side-by-side + sam3d-final-data；`ShapePriorWarmupManager._run` 与 `main.main` 给出 terminal fatal 与 downstream PGID 清理边界。